

Lab 7

Sam Kutsyn, 2581500

EE 446

August 8, 2026

Submission

See submission files in ./submission folder.

Question 1: Model architecture and ensemble flow

Each sample is a 100-time-step window of 6 IMU channels flattened row-major into $100 \times 6 = 600$ input neurons. The same window is fed to three branches that differ only in the input scaling applied to the raw signal: raw (unscaled, m/s^2 and deg/s), standard-scaled (`StandardScaler`, per-channel zero mean / unit variance) and min-max-scaled (`MinMaxScaler`, per-channel to $[0, 1]$).

Per branch (identical topology, three independently trained copies):

- Autoencoder used only for representation learning, 50 epochs, Adam(1e-3), MSE: encoder $600 \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(32, \text{linear})$, decoder $32 \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(600, \text{linear})$. The decoder is discarded after training; only the encoder is kept.
- Classifier on the frozen 32-D latent vector, 50 epochs, Adam(1e-3), categorical cross-entropy: $32 \rightarrow \text{Dense}(20, \text{ReLU}) \rightarrow \text{Dense}(12, \text{Softmax})$.

For deployment each branch is flattened into one Sequential model: $600 \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(32, \text{linear}) \rightarrow \text{Dense}(20, \text{ReLU}) \rightarrow \text{Dense}(12, \text{Softmax})$.

The ensemble contains 7 trained networks: 3 autoencoders (encoders kept), 3 branch classifiers and 1 meta-classifier; at inference time 4 models run (3 flattened branches + meta). Each branch emits a 12-element softmax vector; the three are concatenated into $3 \times 12 = 36$ values in the fixed order [raw | std | minmax]. The stacked meta-classifier is $36 \rightarrow \text{Dense}(24, \text{ReLU}) \rightarrow \text{Dense}(12, \text{Softmax})$, and its argmax is the final activity.

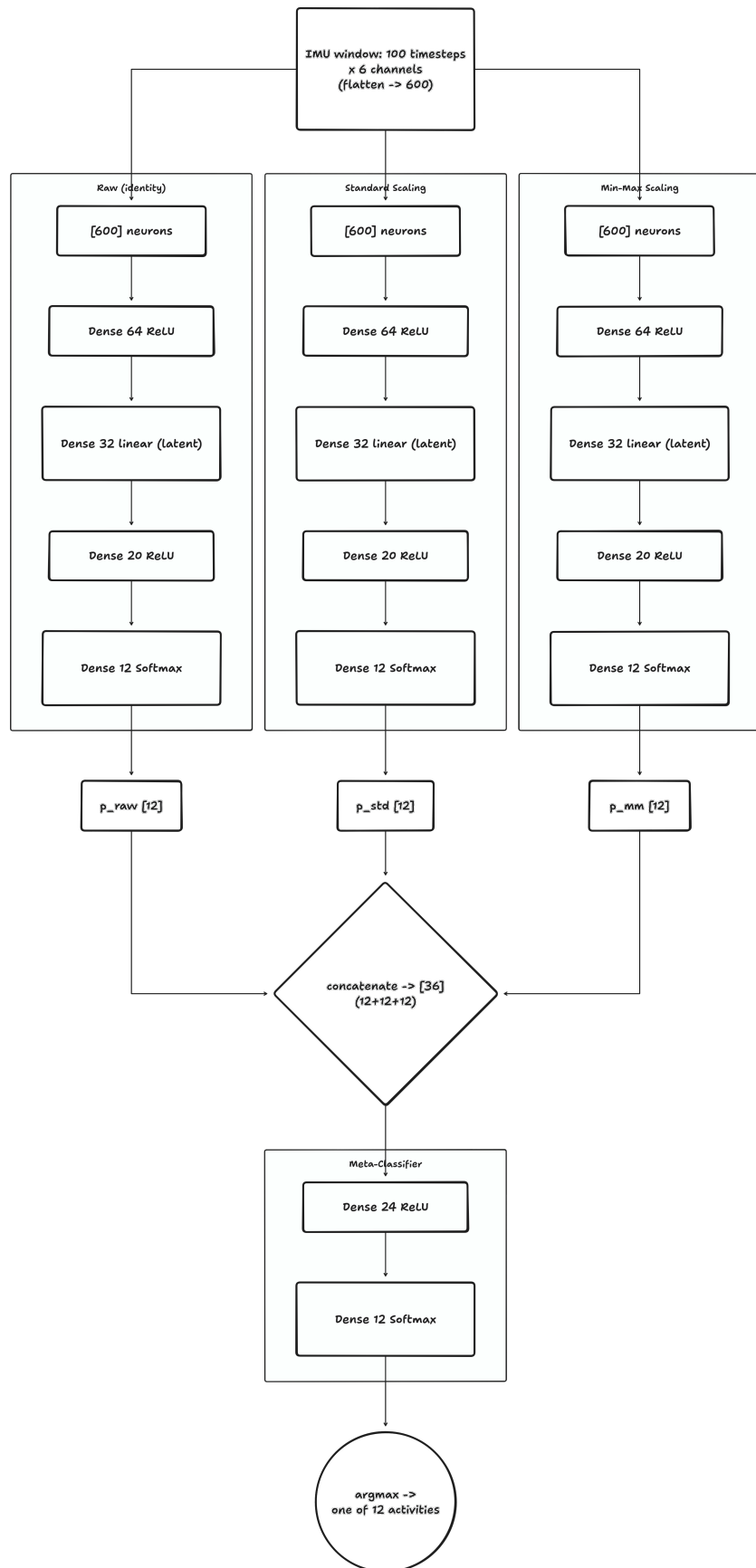


Figure 1: Diagram.

The 12 output classes are the mHealth activities: standing still, sitting and relaxing, lying down, walking, climbing stairs, waist bends forward, frontal elevation of arms, knees bending, cycling, jogging, running, jump front and back (labels shifted from 1-based to 0-based).

Question 2: Pruning and QAT methodology

Pruning

Each flattened branch and the meta-classifier are wrapped with `prune_low_magnitude` and fine-tuned for 5 epochs with the `UpdatePruningStep` callback. The schedule is `PolynomialDecay(initial_sparsity=0.10, final_sparsity=0.80, begin_step=0, end_step=500)`: sparsity ramps smoothly from 10 % to 80 % over the first 500 training steps, so the smallest-magnitude weights are zeroed gradually while the remaining weights keep adapting, instead of one destructive one-shot cut.

Stripping before QAT

`strip_pruning` removes the `PruneLowMagnitude` wrappers, the mask variables and the pruning step counters, leaving a plain Keras model whose kernels simply contain zeros. This is required because `quantize_model` cannot wrap an already-wrapped layer — nesting quantize wrappers around pruning wrappers is unsupported, and the leftover pruning bookkeeping variables would also end up in the converted graph. Stripping keeps the sparsity (the zeros) but drops the machinery.

QAT after pruning

The stripped sparse model is passed through `quantize_model` inside a `quantize_scope()`, which inserts fake-quant nodes on weights and activations, then it is fine-tuned for 5 more epochs. The network therefore learns weights that are already robust to int8 rounding, recovering most of the accuracy that naive post-training quantization would lose.

Why the extra mask enforcement

After stripping, nothing constrains the zeros any more: QAT is ordinary gradient descent, so Adam updates (momentum, weight decay, non-zero gradients) immediately push pruned weights off zero, and the fake-quant rounding can also map near-zero values to a non-zero quantization level. The sparsity would silently evaporate during the 5 QAT epochs. The notebook therefore snapshots the binary masks (`extract_dense_weight_masks, w != 0`) from the stripped model and re-applies them with `MaskEnforcerCallback` after every training batch and every epoch, plus once more after `fit` returns, so the exported model provably keeps ~80 % zeros (`compute_model_sparsity` prints the layer-wise result).

int8 TFLite conversion

The QAT model is converted with `optimizations = [Optimize.DEFAULT]`, `supported_ops = [TFLITE_BUILTINS_INT8]` and `inference_input_type = inference_output_type = tf.int8`, i.e. a full-integer model with no float fallback. A `representative_dataset` of 200 evenly spaced training windows is required because weight ranges are known statically but **activation** ranges are not: the converter runs those samples through the graph to observe each intermediate tensor's dynamic range and derive its scale and zero-point. Without representative data the converter

cannot calibrate activations and would fall back to a dynamic-range or hybrid model, which the TFLITE_BUILTINS_INT8 target forbids.

Why this matters on the Nano 33 BLE Sense

The board has 256 KB SRAM and 1 MB flash with no FPU-friendly float kernels. int8 quantization gives a 4x reduction in weight storage and lets the CMSIS-NN integer kernels run the MACs, which is both faster and lower-energy than float emulation. Measured here, each branch drops from ~169 KB of float32 TFLite (163.7 KB encoder + 5.4 KB classifier) to **45.6 KB** int8, and the meta-classifier from 6.5 KB to **3.6 KB** — all four models together are ~139 KB, which fits in flash alongside a 56 KB tensor arena. Pruning contributes robustness and compressibility (the 80 % zeros compress well in the .cc array and shrink the effective model), while quantization delivers the direct memory and latency win.

Question 3: Arduino deployment and real-world behavior

The screenshot displays the Arduino IDE environment. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for checking, running, and uploading code, along with a dropdown menu currently set to 'Arduino Nano 33 BLE'. The file explorer on the left shows the project files: TinyEnsembleLearning.ino, encoder_clf_minmax_pruned_qat_int8.cc, encoder_clf_std_pruned_qat_int8.cc, and encoder_clf_rav... The main editor window shows the TinyEnsembleLearning.ino file with the following code:

```

17 extern unsigned int encoder_clf_std_pruned_qat_int8_tflite_len;
18
19 extern unsigned char encoder_clf_minmax_pruned_qat_int8_tflite[];
20 extern unsigned int encoder_clf_minmax_pruned_qat_int8_tflite_len;
21
22 extern unsigned char stacked_meta_clf_pruned_qat_int8_tflite[];
23 extern unsigned int stacked_meta_clf_pruned_qat_int8_tflite_len;
24
25 // Activity labels used by the trained models.
26 const char* label_map[12] = {
27     "Standing still",
28     "Sitting and relaxing",
29     "Lying down",
30     "Walking",

```

The Serial Monitor is open at the bottom, showing the output of the program. It includes a message input field, a dropdown for 'Both NL & CR', and a baud rate dropdown set to '115200 baud'. The output text is as follows:

```

Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.2187, 0.4570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0117, 0.0937, 0.21
Standard-scaled model scores: 0.0000, 0.0000, 0.2695, 0.0000, 0.4609, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.2695,
Min-max-scaled model scores: 0.0000, 0.0000, 0.0859, 0.0000, 0.9063, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
Meta-classifier scores: 0.0117, 0.0703, 0.0547, 0.0078, 0.7539, 0.0117, 0.0078, 0.0078, 0.0234, 0.0117, 0.0078
-----
Final prediction: Climbing stairs
Class index: 4
Confidence score: 0.7539
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0430, 0.9570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.00
Standard-scaled model scores: 0.0000, 0.8828, 0.0117, 0.0000, 0.0000, 0.0508, 0.0000, 0.0000, 0.0195, 0.0117,
Min-max-scaled model scores: 0.0000, 0.0156, 0.4453, 0.0469, 0.0391, 0.2617, 0.0781, 0.1133, 0.0000, 0.0000,
Meta-classifier scores: 0.0156, 0.0781, 0.5859, 0.0039, 0.0078, 0.0430, 0.0039, 0.0039, 0.1328, 0.0078, 0.0039
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5859
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0430, 0.9570, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.00
Standard-scaled model scores: 0.0078, 0.3477, 0.0781, 0.0000, 0.1367, 0.1172, 0.0000, 0.0000, 0.0781, 0.0781,

```

The status bar at the bottom indicates 'Arduino Nano 33 BLE on /dev/ttyACM0'.

Figure 2: Serial monitor output with the board resting flat

With the board lying flat and motionless, the sketch collects 100 samples at 50 Hz (2 s per window) and then prints the three branch softmax vectors and the stacked prediction.

Issues:

Unit mismatch on the gyroscope

mHealth stores gyro in deg/s, but the sketch multiplies `readGyroscope()` (deg/s) by `kDegToRad`, so on-board gyro values are ~57x smaller than anything the models saw in training.

Sensor placement / orientation mismatch

The six columns used from the log are the *left-ankle* accelerometer and gyroscope. The training means (`mean(Accel_Y) = -9.15 m/s2`) show gravity on the Y axis, whereas a Nano lying flat on a table puts -9.8 m/s^2 on Z. The static gravity signature is on the wrong axis before any motion is even considered.

Practical improvements:

1. Fix the units and axes first — drop the deg/s to rad/s conversion, and remap or re-orient the board axes so gravity lands on the same channel as in training (or mount the board on the ankle as in the dataset).
2. Collect a small calibration set on the board itself with `Raw_IMU_Recorder`, then either recompute the scaler statistics from that data or fine-tune the models on it; additionally smooth the output with a majority vote over several consecutive windows and reject predictions below a confidence threshold.

Question 4: Deployment behavior on the Arduino

Yes — the deployed ensemble repeatedly predicts the same class even when the board is moved differently, and the meta softmax stays near 1.0 for that class.

The root cause is domain shift: the models were trained on the mHealth subject-6 left-ankle Shimmer channels and are being fed BMI270 data from a hand-held board.

The fix is to align the deployment domain with the training domain (correct units and axes, matching sensor placement) or, better, to retrain on data recorded from the board itself.